Information: Sources & Representation

EC 441 - Introduction to Computer Networking

Boston University

Spring 2026

Learning Objectives

- Define information quantitatively using Shannon entropy
- Calculate entropy of probability distributions
- Distinguish between data and information
- Convert between units (bits, bytes, SI, binary prefixes)
- Explain text encoding (ASCII, UTF-8, base64)
- Understand binary file formats and magic numbers
- Estimate data rates for multimedia

What is Information?

Information is what reduces uncertainty

Surprisal of an event with probability p:

$$I(p) = -\log_2(p) \text{ bits}$$

Examples:

- Coin flip ($p = 0.5$): $I = 1$ bit
- Rolling a 6 ($p = 1/6$): $I \approx 2.58$ bits
- Certain event ($p = 1$): $I = 0$ bits

Shannon Entropy

Entropy = average information per message

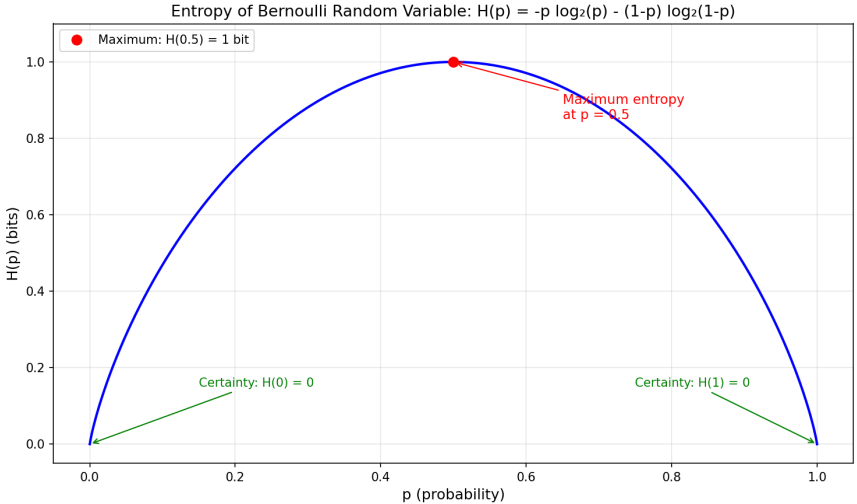
$$H(X) = -\sum_{i=1}^{n} p_i \log_2(p_i) \text{ bits}$$

Represents:

- Minimum bits needed to represent outcome
- Fundamental limit on data compression
- Average amount of information per message

Fair coin: $H = 1$ bit — **Biased coin (0.9/0.1):** $H \approx 0.47$ bits

Entropy Visualization: $H(p)$



- Maximum at $p = 0.5$ (most uncertain)

Units of Information

Basic Units:

- **Bit** (b): Binary digit (0 or 1)
- **Byte** (B): 8 bits
- **Octet**: 8 bits (networking term)

SI Prefixes (base-10):

- Kilo (k): $10^3 = 1{,}000$
- Mega (M): 10^6
- Giga (G): 10^9
- Tera (T): 10^{12}

Used for: Network speeds

Binary Prefixes (base-2):

- Kibi (Ki): $2^{10} = 1{,}024$
- Mebi (Mi): 2^{20}
- Gibi (Gi): 2^{30}
- Tebi (Ti): 2^{40}

Used for: RAM, file sizes

Key: 100 Mb/s = 100,000,000 bits/sec (not 104,857,600!)

Data $\neq$ Information

$$\text{Data size} \geq \text{Information content}$$

- **Information** = actual content (measured in entropy)
- **Data** = how we represent it (measured in bits/bytes)
- Difference = redundancy, overhead, metadata

Example: English text

- Entropy: $\sim$1.5 bits/character
- ASCII: 8 bits/char (19% efficient)
- Compressed: $\sim$2 bits/char (75% efficient)

Tradeoffs: Efficiency vs. Accessibility vs. Reliability

ASCII

American Standard Code for Information Interchange

- 7 bits per character (128 characters)
- Extended ASCII: 8 bits (256 characters)

```
'A' = 0x41 = 65     'a' = 0x61 = 97
'0' = 0x30 = 48     ' ' = 0x20 = 32
```

Key ranges:

- Control: 0x00-0x1F
- Digits: 0x30-0x39
- Uppercase: 0x41-0x5A
- Lowercase: 0x61-0x7A

Unicode and UTF-8

Problem: ASCII only handles English

Unicode: 149,186+ characters

- Code points: U+0041 (A), U+4F60 ()

UTF-8: Variable-length encoding

- 1 byte: ASCII (U+0000 to U+007F)
- 2 bytes: Latin, Greek, Cyrillic, Arabic
- 3 bytes: Asian characters, emoji
- 4 bytes: Rare characters

```
"Hello"      -> 5 bytes
"Hello       " -> 12 bytes (5 + 1 + 6)
```

Base64 Encoding

Purpose: Represent binary data using ASCII printable characters

Alphabet: A-Z, a-z, 0-9, +, / (64 chars = 6 bits each)

Process:

1. Take 3 bytes (24 bits)
2. Split into 4 groups of 6 bits
3. Map to base64 characters

```
Input:  "Man" = 0x4D 0x61 0x6E
Result: "TWFu"
```

Efficiency: 3 bytes → 4 chars (33% overhead)
Uses: Email attachments, embedded images, API tokens

Text File Formats

Different goals → different formats

- **JSON:** Data interchange, self-describing, no schema needed
- **TOML:** Configuration files, supports comments
- **YAML:** Indentation-based, human-friendly configs
- **CSV:** Tabular data, simple but limited
- **XML:** Verbose but flexible, declining use

JSON tradeoff:

- 3-11× overhead vs. binary
- But: human-readable, debuggable, universal support

Magic Numbers (File Signatures)

Bytes at file start that identify format

```
PNG:  89 50 4E 47 0D 0A 1A 0A
JPEG: FF D8 FF
GIF:  47 49 46 38
PDF:  25 50 44 46  (%PDF)
ZIP:  50 4B 03 04  (PK)
ELF:  7F 45 4C 46
```

Purpose:

- Quick file type identification
- Prevents misinterpretation
- Used by `file` command

IEEE 754 Floating-Point

Standard for representing real numbers

Binary32 (float): [Sign:1][Exponent:8][Mantissa:23] = 32 bits

- Range: $\pm 3.4 \times 10^{38}$
- Precision: $\sim$7 decimal digits

Binary64 (double): [Sign:1][Exponent:11][Mantissa:52] = 64 bits

- Range: $\pm 1.7 \times 10^{308}$
- Precision: $\sim$15 decimal digits

Example: $1.5_{10} = 1.1_2 = 1 \times 2^0 + 1 \times 2^{-1}$

Instruction Set Encoding

Principle: Frequent operations → fewer bits (like Huffman coding)

x86 variable-length encoding:

- `NOP`: 1 byte (very common)
- `PUSH EAX`: 1 byte (common)
- `MOV EAX,5`: 5 bytes (less common)

x86 (CISC):

- Variable-length
- Better code density
- Complex decoding

ARM (RISC):

- Fixed 32-bit
- Simple decoding
- Less code density

Network Protocol Formats

Messages aligned to 32-bit boundaries

Similar to file formats:

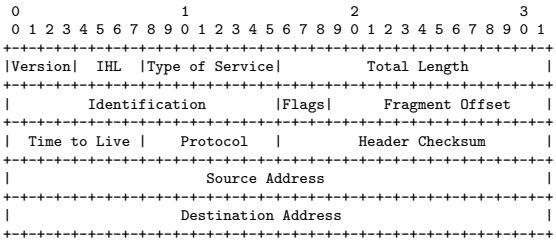
- Magic numbers $\rightarrow$ Version/type fields
- Length fields $\rightarrow$ Total Length, Data Offset
- Checksums $\rightarrow$ Data integrity
- Variable sections $\rightarrow$ Options

Key differences:

- Ephemeral (not stored long-term)
- Strict size limits (MTU $\sim$1500 bytes)
- Performance critical (billions/sec)
- Must maintain backward compatibility

Why 32-bit alignment? Hardware efficiency, simpler parsing

Example: IPv4 Header



- Each row = 32 bits (4 bytes)
- Bit-level fields (Version: 4 bits, Flags: 3 bits)
- Minimum 20 bytes, options extend it

Audio: Analog to Digital

Two steps:

1. **Sampling:** Measure amplitude at regular intervals
   - Sampling rate (Hz): samples/second
   - Nyquist theorem: Must sample at $\geq 2\times$ highest frequency
2. **Quantization:** Round to discrete values
   - Bit depth: bits per sample
   - More bits = less noise

Audio Data Rates

CD Quality:

- 44.1 kHz, 16 bits, stereo
- Data rate: $44{,}100 \times 16 \times 2 \approx 1.4$ Mb/s
- 1 minute $\approx$ 10.5 MB

Voice (telephone):

- 8 kHz, 8 bits, mono
- Data rate: 64 kb/s

Compression:

- Lossless (FLAC): 40-60% of original
- Lossy (MP3): 128-320 kb/s (10-30$\times$ compression)

Images

1920×1080 RGB image:

- Pixels: $1{,}920 \times 1{,}080 = 2{,}073{,}600$
- 24 bits/pixel (8 bits per R, G, B)
- Uncompressed: 6.2 MB

Formats:

- **PNG:** Lossless, 2-4× compression (photos), transparency
- **JPEG:** Lossy, 10-50× compression, no transparency

Typical file sizes (1920×1080):

- PNG: 0.5-3 MB
- JPEG (high): 200-500 KB
- JPEG (medium): 50-150 KB

Video

Uncompressed 1080p @ 30fps:

- 6.2 MB/frame $\times$ 30 fps = 186 MB/s = 1.49 Gb/s
- *Completely impractical!*

Compression techniques:

- **Intra-frame:** Compress each frame (like JPEG) - 10-20$\times$
- **Inter-frame:** Store only differences between frames - 5-10$\times$
- **Total:** 50-200$\times$ typical

Typical compressed rates:

- YouTube 1080p: 3-5 Mb/s
- Netflix 4K: 15-25 Mb/s
- Blu-ray: 20-40 Mb/s

Key Takeaways

1. **Information is quantifiable:** Entropy $H(X)$ = minimum bits
2. **Units matter:** SI vs. binary prefixes
3. **Data $\neq$ Information:** Tradeoffs everywhere
4. **Text encoding:** ASCII $\rightarrow$ UTF-8, Base64 for binary
5. **Binary formats:** Compact but need exact specification
6. **Multimedia:** Compression is essential
   - Audio: 1.4 Mb/s (CD) $\rightarrow$ 128 kb/s (MP3)
   - Video: 1.5 Gb/s (1080p raw) $\rightarrow$ 5 Mb/s (compressed)

Connection to Networking

Why this matters:

- **Bandwidth:** Depends on representation
  - 4K streaming needs 15-25 Mb/s
  - Without compression: impossible!
- **Latency sensitivity:** Voice $<$ 150ms, Video $<$ 400ms
- **Error tolerance:** Text (TCP) vs. Video (UDP OK)
- **Protocol design:** Must handle different data types
  - HTTP for text/JSON/images
  - RTP for real-time audio/video

Next Lecture

Physical Layer: Media and Propagation

- How do we actually send bits?
- Physical media: copper, fiber, wireless
- Speed of light limitations
- Signal propagation and attenuation